

 Introduce Yourself

Navigating The Course

 Levels & Titles

 Beginner Roadmap – Day 1 to AI...

Where This Leads

🔥 DON'T WORRY! You Can't Mess ... 

 Your Claude Code Toolkit

👥 Members' Builds

📍 From Chat to Claude Code: Why ... 

Two Doors: The App vs The Ter...

Installing Claude Code

The Desktop App: No Terminal


```

graph LR
    A[cd — go to your project] --> B[claude — launch Claude Code]
    B --> C[/clear — fresh start]
    C --> D[/model — switch Claude]
    D --> E[/resume — pick up where you left off]
    E --> F[! — quick one-off command]
  
```

```

graph LR
    A[ls] --> B[cd]
    B --> C[git status]
    C --> D[git commit]
    D --> E[npm install]
  
```

YOU DESCRIBE IT. CLAUDE RUNS THE REST.

Quick reminder before we start: this is **optional**. The app covers everything. But if you've decided to keep a foot in the terminal, here's the good news that nobody tells beginners.

To get going:

-  Beginner Prompt Pack 

The Undo Button — You Can't Bre...

-  90 Minutes A Night: The Stolen-... 
-  (Optional) Inside the Terminal: A ... 
-  (Optional) The Few Terminal Co... 

 (Optional) The Few Terminal Commands You'll Eve - STAGE 1 - START HERE · Claude Code Club

- `claude` launches Claude Code right there in the terminal. That's the magic word. From this point on you're just talking to Claude again, same as the app.

Once Claude Code is open, a few slash-commands earn their keep:

- `/clear` wipes the current conversation and starts fresh. Use it when you switch to a totally new task and don't want Claude carrying over old context.
- `/model` lets you switch which Claude you're using. Worth knowing when you want a faster or more powerful one for a specific job.
- `/resume` reopens a past conversation right where you left it. This one is genuinely great: close your laptop, come back tomorrow, type `/resume` , and your project picks up mid-thought.
- `!` lets you run one quick computer command without leaving Claude. Type `!` then your command. You'll rarely need it, but it's there.

That's the real list. Everything else, you describe and Claude does.

What you'll see Claude run (so you can read along)

You won't type these, but you'll watch Claude run them, and it helps to know what they mean so the screen isn't a mystery:

- `ls` lists files. `cd` moves between folders.
- `git status` checks what's changed. `git commit` saves a checkpoint of your work.
- `npm install` adds a tool a project needs.

When you see these scroll by, that's just Claude doing the plumbing. Read them like subtitles, not like homework.

When it's actually worth opening the terminal

The app is home base. But a few specific moments are smoother in the terminal:

- **Long or background jobs.** If something needs to run for a while, the terminal handles it without tying up your window.



- **When the app is being weird.** If the desktop app stalls, launching `claude` in a terminal is a reliable plan B.

None of these are required. They're just the times a member might reach for the terminal on purpose.

If none of this calls to you, close the terminal and stay in the app. You are not missing anything. The people shipping the most in this community are split right down the middle between app and terminal, and the builds look identical.

✓ **Done when:** you know the short list you actually type (`cd` , `claude` , `/clear` , `/model` , `/resume` , and `!`), and the few moments the terminal is worth opening.